UQÀM | Faculté des sciences

Projet 1 — Application de Gestion de Pizzeria

Maîtrise en Génie Logiciel – MGL7030

Équipe:

Amadou Sidy Gueye

Rapha Ondobo Mvolo

Remis le : 18 février 2026

Table des matières

Introduction 3

1. Question 1 (2%) 3

Suppression du visuel par défaut des éléments <ul> et <li> 3

2. Question 2 (4%) 4

Sélection d'une seule garniture viande, plusieurs légumes et un seul format 4

3. Question 3 (4%) 5

3.1 Ajout d'une commande sans modifier les entrées existantes 5

4. Question 4 (2%) 6

Pourquoi préférer addEventListener aux attributs HTML (onclick, onchange, etc.) 6

5. Question 5 (18%) 7

Implémentation du bouton « Supprimer » pour les commandes dynamiques 7

Conclusion 9

INTRODUCTION

Ce rapport accompagne le code source du Projet 1 du cours MGL7030. Il documente les choix de conception et d'implémentation retenus pour la couche présentation (frontend) de l'application de gestion de Pizzeria. L'application a été développée en HTML, CSS et JavaScript natifs, sans l'utilisation de bibliothèques ou Framework externes.

Les sections ci-dessous répondent aux questions du rapport telles que définies dans l'énoncé du projet.

---

1. QUESTION 1 (2%)

Suppression du visuel par défaut des éléments <ul> et <li>

Les listes HTML non ordonnées (<ul>) affichent par défaut un point (bullet point) avant chaque élément <li>. Ce comportement visuel est géré par la propriété CSS list-style. Pour le supprimer, la règle CSS suivante a été appliquée :

```
.menu-items {
    list-style: none;
    padding: 0;
    margin: 0;
}
```

Explication de l'approche :

- list-style: none; : Cette propriété CSS contrôle l'apparence du marqueur de liste. En lui assignant la valeur none, on indique au navigateur de ne pas afficher de marqueur (ni point, ni numéro, ni aucun autre symbole) devant les éléments de liste.
- padding: 0; margin: 0; : Ces deux déclarations suppriment l'indentation et les espaces extérieurs que les navigateurs ajoutent par défaut aux éléments <ul>, ce qui permet d'aligner visuellement les items avec le reste du contenu.

Cette approche fonctionne parce que les propriétés CSS héritées ou les styles par défaut du navigateur (user-agent stylesheet) peuvent être explicitement remplacées par des règles CSS

personnalisées. En ciblant la classe .menu-items appliquée sur la balise <ul>, on remplace le style du navigateur uniquement pour cette liste, sans impacter les autres listes de la page.

---

2. QUESTION 2 (4%)

Sélection d'une seule garniture viande, plusieurs légumes et un seul format

Le comportement de sélection différencié entre les garnitures viande, les légumes et le format est assuré par le choix du type d'élément de formulaire HTML utilisé pour chaque catégorie.

1. Garniture viande — une seule sélection possible :

Les garnitures de type viande (Pepperoni, Poulet, Crevette, Viande) utilisent des boutons radio (<input type="radio">). Tous ces boutons partagent le même attribut name="garniture". Le comportement natif des boutons radio en HTML impose qu'un seul bouton peut être sélectionné à la fois parmi ceux qui partagent le même name. Ainsi, choisir une nouvelle viande désélectionne automatiquement la précédente.

```
<input type="radio" name="garniture" id="pepperoni" value="Pepperoni">
<input type="radio" name="garniture" id="poulet" value="Poulet">
```

2. Garnitures légumes — sélections multiples possibles :

Les légumes (Poivron, Oignon, Olives, Champignon) utilisent des cases à cocher (<input type="checkbox">). Chaque case à cocher est un contrôle indépendant. Il n'y a pas de contrainte de sélection unique liée au name pour les checkbox. L'utilisateur peut donc cocher autant de légumes qu'il le souhaite simultanément.

```
<input type="checkbox" name="legume" id="poivron" value="Poivron">
<input type="checkbox" name="legume" id="oignon" value="Oignon">
```

3. Format — une seule sélection possible :

Les formats (Petite, Moyenne, Grande) utilisent également des boutons radio partageant le même attribut name="format". Le même mécanisme que pour les viandes s'applique : un seul format peut être sélectionné à la fois.

```
<input type="radio" name="format" id="petite" value="petite">
<input type="radio" name="format" id="moyenne" value="moyenne">
```

En résumé, la distinction radio vs checkbox est le mécanisme fondamental qui permet de contrôler le nombre de sélections autorisées dans chaque catégorie, sans avoir recours à du JavaScript pour la gestion de la sélection.

---

3. QUESTION 3 (4%)

3.1 Ajout d'une commande sans modifier les entrées existantes

Pour ajouter une nouvelle entrée dans la section « Commandes » sans altérer les lignes déjà présentes, on utilise la méthode DOM insertAdjacentHTML() ou insertBefore() sur le corps du tableau (<tbody>), combinée à la création dynamique d'éléments HTML.

L'approche retenue repose sur les principes suivants :

- Création d'un nouvel élément <tr> : Au lieu de réécrire le innerHTML complet du tableau (ce qui détruirait toutes les entrées existantes), on crée un nouvel élément de ligne DOM avec document.createElement('tr').
- Insertion en première position : La nouvelle commande est insérée en tête de liste grâce à tbody.insertBefore(newRow, tbody.firstChild). Cela place la commande la plus récente au-dessus sans toucher aux autres noeuds du DOM.
- Isolation des modifications : Chaque ligne est un noeud DOM indépendant. Manipuler ou insérer un noeud ne modifie pas les propriétés ou gestionnaires d'événements des noeuds voisins.

Exemple simplifié de l'implémentation :

```
const tbody = document.querySelector('.command-item');
const newRow = document.createElement('tr');
newRow.innerHTML = `<td>${date}</td><td>${format}</td>...`;
tbody.insertBefore(newRow, tbody.firstChild);
```

Cette approche respecte l'exigence de l'énoncé : « L'ajout ne doit pas modifier le HTML des autres entrées déjà présentes ». En travaillant au niveau des noeuds DOM individuels plutôt qu'en réécrivant le contenu HTML complet du conteneur, l'intégrité des entrées existantes est garantie.

---

4. QUESTION 4 (2%)

Pourquoi préférer addEventListener aux attributs HTML (onclick, onchange, etc.)

L'utilisation de addEventListener est préférable aux attributs HTML d'événement pour plusieurs raisons fondamentales :

- Séparation des préoccupations (Separation of Concerns) : addEventListener permet de conserver le code JavaScript dans les fichiers .js, complètement séparé du HTML. Les attributs inline (onclick="...") mélangent la logique applicative avec la structure de la page, rendant le code plus difficile à lire, maintenir et déboguer.
- Ajout de plusieurs gestionnaires sur un même événement : Un élément ne peut avoir qu'un seul attribut onclick, ce qui limite la modularité. Avec addEventListener, on peut attacher plusieurs écouteurs au même événement sans que l'un écrase l'autre.

```
// Les deux handlers s'exécutent tous les deux
btn.addEventListener('click', handlerA);
btn.addEventListener('click', handlerB);
```

- Contrôle du cycle de vie des événements : addEventListener permet de supprimer un gestionnaire avec removeEventListener, offrant un contrôle précis sur la durée de vie des écouteurs. Cela est essentiel pour éviter les fuites mémoire dans les applications dynamiques.
- Portée et sécurité : Les attributs inline s'exécutent dans la portée globale, ce qui peut créer des conflits avec d'autres variables ou fonctions. addEventListener s'intègre proprement dans la portée du module JavaScript, surtout avec l'utilisation du type="module" dans la balise script.
- Capture et propagation (bubbling/capturing) : addEventListener accepte un troisième argument pour contrôler la phase de capture ou de bouillonnement des événements, ce que les attributs inline ne permettent pas.

En résumé, addEventListener est la méthode recommandée par les standards web modernes car elle favorise un code plus propre, modulaire et maintenable.

5. QUESTION 5 (18%)

Implémentation du bouton « Supprimer » pour les commandes dynamiques

5.1.1. Création dynamique du bouton « Supprimer »

À chaque clic sur le bouton « Commander », une nouvelle ligne de tableau est créée programmatiquement. Un bouton « Supprimer » est inclus dans cette ligne au moment de sa création, via la propriété innerHTML du nouvel élément <tr> :

```
const newRow = document.createElement('tr');
newRow.innerHTML = `
  <td>${dateStr}</td>
  <td>${format}</td>
  <td>Viande</td>
  <td>${garnitures}</td>
  <td>${prix.toFixed(2)}</td>
  <td><button class='btn-supprimer'>Supprimer</button></td>
`;
```

Le bouton est donc créé à la volée pour chaque commande, sans qu'il existe dans le HTML statique initial. Il est immédiatement disponible dans le DOM dès l'insertion de la ligne.

5.1.2. Identification et suppression de la pizza correspondante

Pour supprimer uniquement la ligne associée au bouton cliqué, on exploite la relation parentale entre le bouton et sa ligne (<tr>). Après avoir récupéré le bouton via querySelector sur la nouvelle ligne, on lui attache un gestionnaire d'événement :

```
const deleteBtn = newRow.querySelector('.btn-supprimer');
deleteBtn.addEventListener('click', function() {
    newRow.remove();
    updateTotal();
```

```
});
```

La méthode newRow.remove() supprime directement le noeud DOM de la ligne du tableau. Grâce à la fermeture (closure) JavaScript, la variable newRow est capturée dans le scope de la fonction de rappel, ce qui permet d'identifier et de supprimer précisément la bonne ligne sans recourir à des identifiants uniques ou à des attributs data supplémentaires.

Une alternative consiste à remonter l'arborescence depuis le bouton :

```
deleteBtn.addEventListener('click', function(e) {
    e.target.closest('tr').remove();
    updateTotal();
});
```

5.1.3. Évitement de la duplication des gestionnaires d'événements

Notre approche garantit l'absence de duplication de gestionnaires pour plusieurs raisons :

- Un gestionnaire par bouton : Le addEventListener est appelé une seule fois par bouton, au moment de la création de la ligne. Il n'est jamais réappliqué ni re-déclaré sur le même bouton.
- Pas de délégation sur le conteneur global : Contrairement à une approche par délégation d'événements sur le tbody (où un seul handler écoute tous les clics et filtre la cible), notre approche affecte directement un listener au bouton concerné. Cela évite les risques d'accumulation de listeners si la même fonction était répétitivement réenregistrée sur un noeud parent.
- Pas de re-rendu du DOM : En n'utilisant pas innerHTML = '...' sur le conteneur complet pour ajouter une commande, on évite que les boutons existants soient recréés et se retrouvent avec des gestionnaires dupliqués ou perdus.
- Nettoyage automatique à la suppression : Lorsque la ligne est retirée du DOM via newRow.remove(), le noeud et tous ses descendants (y compris le bouton et son gestionnaire d'événement) sont détruits. Le garbage collector de JavaScript peut alors libérer la mémoire associée, ce qui maintient les performances de l'application même avec de nombreuses commandes.

Cette implémentation est donc à la fois correcte, performante et conforme aux bonnes pratiques de développement JavaScript moderne.

CONCLUSION

Ce rapport a présenté les choix techniques réalisés pour l'implémentation du frontend de l'application de gestion de Pizzeria dans le cadre du cours MGL7030. Les solutions retenues s'appuient sur des mécanismes natifs du HTML, CSS et JavaScript, sans recourir à des bibliothèques externes.

Les principaux points abordés sont : la suppression du style par défaut des listes via CSS, le contrôle du comportement de sélection des formulaires grâce aux types radio et checkbox, la manipulation ciblée du DOM pour l'ajout et la suppression de commandes, ainsi que les avantages de addEventListener sur les attributs inline pour une architecture frontend propre et maintenable.